UNIVERSITY OF MINES AND TECHNOLOGY (UMaT)

FACULTY OF COMPUTING AND MATHEMATICAL SCIENCES

DEPARTMENT OF CYBERSECURITY AND INFORMATION SYSTEMS

ACADEMIC YEAR: 2025/2026, Semester 2

NAME: Abdullah Armiyao

INDEX NUMBER: FCM.41.018.068.23

DEGREE: BSc Cybersecurity

COURSE: Cybersecurity Lab Work II (CY384)

PROJECT TITLE: Adaptive Two-Stage Framework for Near Real-Time DDoS Mitigation Using Behavioral Traffic Analysis

1. ARTEFACT DESCRIPTION

1.1 Relation to other works:
Traditional distributed denial-of-service (DDoS) mitigation systems heavily depend on static, hard-coded thresholds that fail to distinguish malicious floods from legitimate, high-volume traffic surges, better known as "flash crowds". This limitation usually results in severe false-positive blocks, taking down critical web infrastructure during high-value business events. Off late, network security frameworks emphasize the need of multi-stage defensive layouts to protect online services without introducing crippling packet-processing overhead. The system architecture of this project draws inspiration from the two-stage adaptive detection pipeline logic proposed by Bai *et al. (*2026). Also, the processing model builds upon the validation parameters established by Abiramasundari *and Ramaswamy (*2025), who demonstrated the superior classification accuracy of Random Forest models on structured, behavioral tabular features. This project is to design and implement an automated inline network security software deployed on linux gateway bridges. This project uses a continuous statistical pre-filtering algorithm, together with Machine Learning (ML) algorithms, to block malicious traffic at the kernel level. The software prevents incoming volumetric attacks from exhausting gateway hardware resources by executing line-rate packet drops directly within kernel-space. By deploying this two-stage architecture, the system shifts heavy processing burden away from inline machine learning inference, making gateway nodes maintain optimal throughput during regular operations while simultaneously remaining responsive during attacks. The project acts as a defensive gateway that bridges the gap between theoretical academic detection models and practial, low-resource network defenses.

1.2 Aim & objectives of the project:

The main objective of this research is to design, develop, and evaluate a deployable, lightweight, two-stage adaptive DDoS mitigation gateway system that operates in near real time on Linux hardware, minimizing computational overhead while maximizing resilience against false-positive triggers during legitimate flash crowds.

1.3 List of features that the artefact will include:

1. *Bivariate Streaming Anomaly Scoring via Welford Dynamics*: Combines Shannon source IP entropy measurements with Packet Rate Exponentially Weighted Moving Averages (EWMA). These streaming metrics are fed directly into Welford's algorithm for online variance tracking, computing a memory-optimized running traffic mean ($\boldsymbol{\mu}$) and standard deviation ($\boldsymbol{\sigma}$) in O(1) constant time.

2. *On-Demand Classifier Activation*: Keeps the machine learning layer inactive until the current window's metrics breach a dynamic, Welford-calculated confidence boundary ($\boldsymbol{\mu}$+ 2$\boldsymbol{\sigma}$), completely eliminating historical database logging requirements and saving host CPU power.

3. *Collinearity-Resilient Machine Learning*: Uses an optimized Random Forest architecture tracking 5 normalized behavioral features: Source IP Entropy, Packet Rate Deviation, Packet Size Variance, SYN/ACK Ratio, and Layer 4 Protocol Distribution.

4. *Kernel-Level Constant Time O(1) Dropping*: Injects validated malicious IPs into high-performance `ipset` hash tables linked to Netfilter, achieving immediate high-speed packet rejection.

5. *Closed-Loop Feedback Recalibration*: When Stage 2 verifies an anomaly as an organic flash crowd, it signals Stage 1 to adaptively widen its variance thresholds, preventing repetitive false-alarm cycles.

6. *Automated Graceful Degradation*: If classification outputs are highly ambiguous ($0.4 < P < 0.6$), the system safely falls back to a monitoring-and-alerts-only state to protect service availability.

7. *Asymmetric Temporal Boundary Decay (k-Decay)*: To prevent the gateway from becoming permanently "blinded" after adapting to a legitimate flash crowd, the system uses an automated time-decay function. After a closed-loop threshold expansion, the variance multiplier (k) exponentially decays back to its strict baseline value at a rate proportional to time t since the last anomaly, ensuring the pre-filter re-adjusts itself against subsequent stealth attacks.

1.4 Added value that the project provides:

1. *State Exhaustion Protection for Target Backends*: Traditional firewalls inspect traffic but still pass connections to the application layer. By utilizing the Linux kernel's ipset hash tables linked to Netfilter, this project drops malicious packets at the lowest possible layer of the operating system stack (Layer 3/4). This guarantees that backend application infrastructure is completely shielded from connection-tracking table overflows and TCP socket state exhaustion.

2. *Decoupled Compute Conservation*: In standard machine learning security systems, running constant matrix multiplication on every single incoming packet will max out gateway CPU resources and cause massive latency for legitimate users. This project introduces computational isolation. During normal traffic conditions, the heavy Random Forest machine learning engine remains entirely dormant, consuming 0% CPU. Host compute resources are only allocated for complex behavioral inference when the lightweight statistical pre-filter explicitly flags an active anomaly.

3. *Self-Sensitizing Autonomy via k-Decay*: Most adaptive threshold systems require a human network administrator to manually reset security baselines after a traffic surge. By integrating the asymmetric temporal boundary decay (k-decay) routine, this product introduces operational self-healing. The system automatically re-sensitizes its own statistical filters immediately after a traffic spike passes, maintaining a high security posture without requiring persistent, manual administrative oversight.

4. *Low-Latency Edge Execution*: Because the entire architecture runs inline on a local Linux bridge (br0), detection and mitigation happen directly at the network perimeter. This completely eliminates the multi-second back-and-forth routing latencies associated with forwarding traffic to external, cloud-based scrubbing networks, keeping local service response times efficient.

1.5 Intellectual challenges involved:

Mathematical & Algorithmic Challenges

1. *Real-Time Stream Vectorization under Saturation Constraints*: Resolving the extreme computational overhead required to extract multi-layered statistical metrics from a high-volume, live network interface inside user-space Python, without inducing kernel ring-buffer overflows or packet drops at the sniffer layer during peak volumetric stress.

2. *Mitigation of Adversarial Threshold Steering*: Countering the vulnerability where a sophisticated attacker uses progressive, slow-drip traffic escalations over an extended temporal window to deliberately shift Welford’s running mean and standard deviation. This skews the statistical filter baseline to absorb a subsequent massive attack wave undetected. This is a challenge directly mitigated in this architecture via the inclusion of an automated k-decay function.

3. *Multi-Variable Precision-Recall Optimization*: Calibrating and balancing the closed-loop threshold expansion boundaries to ensure the gateway reacts with sub-second sensitivity to true

volumetric attacks (high recall/sensitivity) while remaining resilient enough to prevent catastrophic false-positive blocks of valid traffic surges (high precision).

4. *Compressing Feature Collinearity and Concept Drift in Static Inference*: Optimizing a low-dimensional, 5-feature vector space for the Random Forest model that resists classification degradation over time when faced with changing network patterns, ensuring it maintains high accuracy across varying network environments without demanding a heavy online retraining loop.

Systems & Infrastructure Challenges

5. *Kernel-to-User Space Inter-Process Communication (IPC) Processing Latency*: Minimizing the communication and processing delay when transferring flagged malicious IP addresses from Python user-space inference down into kernel-space Netfilter ipset tables, ensuring the system achieves immediate packet rejection before network buffers saturate under intense hping3 floods.

6. *Memory-Footprint Constraints on Streaming Connection State Data*: Designing highly optimized, O(1) sliding-window data structures that can continuously compute connection metrics like SYN/ACK ratios and protocol distributions for thousands of concurrent sessions without inducing memory leaks or system instability.

7. *Non-Disruptive Inline Bridge Fail-Safe Orchestration*: Engineering a multi-threaded execution loop with an independent heartbeat monitor to ensure that if the main Python script encounters a critical runtime exception, memory fault, or buffer starvation, the gateway gracefully degrades to a passive fail-open state, thereby preserving uninterrupted network connectivity while generating administrator alerts.

2. METHODOLOGY TO BE USED TO REALIZE THE ARTEFACT

2.1 Approach to be employed to develop the project:

Using the V-Model methodology, the architecture establishes a direct, parallel relationship between design abstraction and empirical testing. The operational execution of this methodology is divided into four phases:

Phase 1: Virtualized Topology Provisioning (The Design Baseline)

- Development Approach: The project begins by setting up a fully isolated, zero-trust network sandbox inside a Proxmox VE hypervisor node. This environment isolates three distinct virtual machines (VMs): an *Attacker VM* configured to generate malicious traffic, a *Sensor VM* configured as a transparent Layer 2 Linux bridge (br0) running the artifact, and a target *Victim VM* hosting an unhardened Nginx web server.
- Testing & Evaluation Plan: This phase is validated using Baseline Verification. Before any defense scripts are active, continuous ICMP ping loops and standard HTTP requests are routed from the Attacker node to the Nginx server through the inactive bridge. This baseline guarantees 100% network transparency, proving the bridge does not induce packet loss during normal routing operations.

Phase 2: Live Ingress Capture & Vectorization (The Data Pipeline)

- Development Approach: A real-time data ingestion engine is built within user-space Python on the gateway bridge using low-level network hooks. The engine continuously captures passing ethernet frames, dissects packet headers via Scapy, and partitions raw traffic into discrete, rolling computation windows of 50 packets.
- Testing & Evaluation Plan: This phase is evaluated via Stream Accuracy Profiling. Synthetic packet bursts are sent through the gateway to verify that the Python script accurately parses Layer 3/4 metadata (Source IPs, TCP flags, and protocol types). The sliding-window logs are audited to ensure that streaming metrics like Shannon Source IP Entropy and packet velocity EWMA are computed accurately on the fly without inducing packet drops or kernel ring-buffer overflows.

Phase 3: Offline Model Synthesis & Validation (The Intelligence)

- Development Approach: To create the machine learning classification layer, controlled data-gathering runs are executed within the sandbox. The gateway logs traffic profiles during simulated baseline operations, intense hping3 volumetric floods, and heavy multi-threaded flash crowds. This localized tabular data is cleaned, normalized, and passed to a supervised Random Forest algorithm via scikit-learn. Feature weights are audited to eliminate feature collinearity, and the optimized model is exported as a serialized binary.
- Testing & Evaluation Plan: The classifier is evaluated using an Offline Performance Matrix. The model is tested against a hidden 30% data partition split. The target metric required to pass this boundary is an F1-Score greater than or equal to 0.98, ensuring an ultra-high precision rate that prevents classification degradation (concept drift) before the binary is introduced to the live inline environment.

Phase 4: Two-Stage Deployment & Closed-Loop Evaluation (The Mitigation Engine)

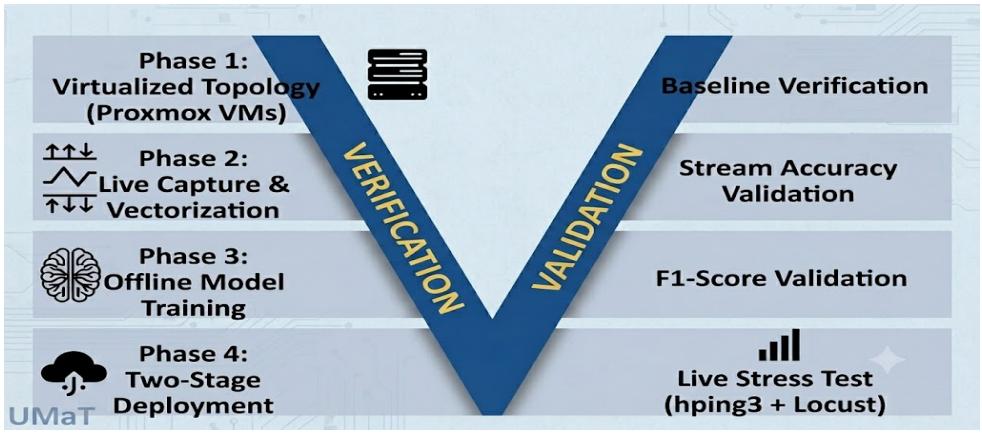
- Development Approach: The final runtime architecture is assembled on the live gateway. The Stage 1 statistical engine constantly monitors traffic streams using Welford’s algorithm

to maintain a dynamic anomaly boundary. When an anomaly trips this threshold, the script wakes up the Stage 2 Random Forest model.

- If the model determines the anomaly is an attack, the script bypasses slow user-space tools and instantly pushes the malicious IP address into kernel space using constant-time O(1) Linux ipset hash tables for line-rate packet rejection.
- If it determines it is a legitimate flash crowd, a closed-loop feedback path widens the Welford boundary, allowing the traffic to pass while triggering an asymmetric temporal k-decay routine to automatically re-sensitize the filter over time.

- Testing & Evaluation Plan: The entire integrated system is subjected to a live, multi-threaded stress test. A high-velocity hping3 flood is launched concurrently with a massive wave of legitimate HTTP traffic simulated via the Locust framework. The ultimate real-world performance of the artifact is empirically evaluated across four distinct live metric vectors:
  - Mitigation Latency: The exact number of milliseconds elapsed between the onset of an attack and the enforcement of the kernel-level ipset drop rule.
  - False-Positive Block Rate: The percentage of legitimate, simulated student HTTP requests mistakenly blocked during a high-volume traffic rush.
  - Inference CPU Conservation: Benchmarking gateway CPU usage to prove that host machine resources remain negligible during peacetime and only spike during an active validation crisis.
  - State Exhaustion Resistance: Monitoring connection tracking tables and socket availability on the backend Nginx web server to verify that the inline gateway effectively shields the backend target from crashing under maximum volumetric stress.

The image below is a graphical representation of the V-Model Methodology which will be used:



2.2 Appropriateness and suitability of this approach for realising the artefact:

This approach is suited for realizing this artifact because it prioritizes live behavioral execution over passive software simulation.
Evaluating an adaptive network security architecture strictly using static, pre-collected datasets is highly insufficient for a deployment-grade gateway. Static data science environments fail to capture

critical operational realities, such as hardware interrupt scheduling disruptions, kernel ring-buffer overflows under line-rate stress, and user-space to kernel-space IPC processing bottlenecks.

By staging the entire project within a live, hardware-emulated network bridge, this approach provides three distinct architectural validations that prove its suitability:

1. Computational Feasibility on Resource-Constrained Hardware

The lightweight Stage 1 statistical pre-filter runs constantly using Welford’s algorithm, which computes a true running variance using a negligible mathematical footprint. This keeps host CPU consumption near 0% during normal traffic conditions, reserving the resource-intensive Stage 2 Random Forest inference model exclusively for active mitigation crises.

2. Absolute Mitigation Efficiency and State Exhaustion Protection

This approach binds the Python decision logic directly to the Linux kernel using Netfilter ipset hash tables. When a malicious IP is identified, the system drops the attacking packets at the lowest possible layer of the operating system stack. This explicitly shields backend web application infrastructure from socket state exhaustion and connection-tracking table overflows.

3. Algorithmic Resilience Against Dynamic Network Variables

Rather than relying on static thresholds that cause false-positive blocks during registration rushes, this methodology tests the closed-loop feedback path and the asymmetric temporal k-decay routine. This proves the system can automatically expand its boundaries to let lot of legitimate traffic through and immediately re-sensitize its defenses once the surge passes, without requiring manual administrative intervention

3. HOW THE PROJECT RELATES TO MY DEGREE COURSE AND BUILDS UPON THE KNOWLEDGE I HAVE ACQUIRED

3.1 Aspects of the project that correlate with knowledge and skills acquired from my course of study:

With this project, I get to integrate technical domains I picked up over my years in this school, throughout the cybersecurity programme. With my knowledge and skill in systems and networking which I picked up from previous semester courses like, Systems and Network Administration, and Linux Fundamentals, I get to dissect transport-layer packet headers, diagnose protocol flag distributions, identify asymmetry in connection counts, and understand the core physics behind host state-exhaustion vectors. Also, with the knowledge and skill I gained from Operating Systems and Systems Programming, I get to developing multi-threaded packet management software using Python sockets, manage Linux network bridges, and program kernel-level automated controls using ipset and iptables. Thanks to AI in Cybersecurity, Cyber Threat Intellingence, and Information theory, I will be running data preprocessing pipelines, establishing balanced traffic profiles, optimizing decision boundaries for tree-based algorithms, and resolving precision-recall trade-offs within serialization environment

4. RESOURCES

4.1 Resources required to develop the artefact:

To successfully implement, deploy, and evaluate this project, it requires a specialized stack spanning physical virtualization hosts, systems programming frameworks, statistical analysis libraries, and multi-threaded adversarial simulation tools.

1. Virtualization & Hardware Infrastructure: A dedicated host bare-metal or hypervisor machine running Proxmox VE to isolate and orchestrate three distinct network nodes.
2. Core Language & Operating System Environments:
    - Linux Ubuntu/Fedora Server: Acting as the foundational kernel space operating system environment for the gateway.
    - Python3: The primary user-space language used to write the feature vectorization, streaming mathematics, machine learning inference, and closed-loop control orchestration logic.
3. In-Line Traffic Interception & Networking Stack:
    - libpcap-dev & Pcapy: Low-level C-based packet capture libraries used to tap into the kernel ring buffer for raw frame ingestion.
    - Scapy: Used within the user-space Python runtime for deep packet inspection, packet metadata slicing, and header parsing.
    - Linux Netfilter Utilities (iptables & ipset): The engine used to maintain hash tables containing banned source IP addresses, executing line-rate packet rejection entirely within kernel space.
4. Analytics, Data Processing & Machine Learning Libraries:
    - NumPy: Utilized for high-speed vectorization, Shannon Source IP entropy calculation loops, and managing windowed array chunks without inducing processing lag.
    - Pandas: Employed for transient data structuring and structured frame evaluation before machine learning scoring.
    - Scikit-learn: The baseline data science library utilized to load, parse, and execute prediction matrices on the pre-compiled behavioral Random Forest binary.
5. Adversarial Synthesis & Empirical Evaluation Frameworks:
    - hping3: A command-line network auditing utility used to generate high-velocity, stateless Layer 4 volumetric attacks to calculate mitigation latency.
    - Locust Framework: A modern, distributed, multi-threaded performance testing framework used to generate valid HTTP request patterns modeled on real high traffic trends, validating the precision of the closed-loop feedback and the threshold stability of the k-decay mechanism during flash crowds.
6. Academic Context & Literature Foundations:

- SpringerLink | https://link.springer.com/article/10.1186/s42400-025-00414-0
- IETF RFC Repository | https://datatracker.ietf.org/doc/rfc4987/
- NIST SP Repository | https://www.nist.gov/privacy-framework/nist-sp-800-61
- Journal of Algorithms (Elsevier) | https://www.sciencedirect.com/journal/journal-of-algorithms
- Nature Scientific Reports | https://www.nature.com/articles/s41598-024-84879-y

4.2 References:

1. T. Bai, Y. Liu, Y. Gao, and Y. Zhou, “ATS-DTA: Adaptive two-stage DDoS detection with dynamic threshold adjustment in SDN networks,” Cybersecurity, vol. 9, Article 12, Jan. 2026, doi: 10.1186/s42400-025-00414-0.

2. S. Abiramasundari and V. Ramaswamy, “Distributed denial-of-service (DDoS) attack detection using supervised machine learning algorithms,” Scientific Reports, vol. 15, Article 13098, 2025, doi: 10.1038/s41598-024-84879-y.

3. E. Cohen and M. Strauss, “Maintaining time-decaying stream aggregates,” Journal of Algorithms, vol. 51, no. 2, pp. 245–267, May 2004, doi: 10.1016/j.jalgor.2003.11.004.

4. W. Eddy, “TCP SYN Flooding Attacks and Common Mitigations,” RFC 4987, IETF, Aug. 2007. Available: https://www.rfc-editor.org/rfc/rfc4987

5. National Institute of Standards and Technology, “Computer Security Incident Handling Guide,” NIST Special Publication 800-61 Rev. 2, U.S. Department of Commerce, Gaithersburg, MD, Aug. 2012.

SIGNATURE: DATE: 14th June 2026